

# SMARTE MÜLLTONNEN- AUTONOMIE

Ein interdisziplinäres Projekt zur intelligenten  
Abfallwirtschaft

Daniel Tüben | Christoph Hardes | Philipp Zajackowski

Master Digitale Technologie

Betreut durch:

Prof. Dr. Frank Hellweg | Prof. Dr. Jens Bechthold | Prof. Markus Strick

Januar 2026





# SMARTE MÜLLTONNEN- AUTONOMIE

Ein interdisziplinäres Projekt zur intelligenten  
Abfallwirtschaft

Daniel Tüben | Christoph Hardes | Philipp Zajackowski

Master Digitale Technologie

Betreut durch:

Prof. Dr. Frank Hellweg | Prof. Dr. Jens Bechthold | Prof. Markus Strick

Januar 2026



# DAS PROBLEM – UND WARUM WIR ES GELÖST HABEN

## Herausforderung der traditionellen Abfallwirtschaft:

- **Ineffiziente Routenplanung:**  
Fahrer fahren feste Routen, auch wenn Tonnen nicht voll sind
- **Unplanbare Leerungskosten:**  
Keine Echtzeitinformation über Füllstand
- **Fehlende Datengrundlage:**  
Keine Insights zur Nutzung und Optimierungspotentialen

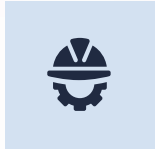
## Unsere Lösung: Die Smarte Mülltonne

- ✓ **Autonome Füllstandserkennung** → Echtzeit-Datenfluss
- ✓ **Intelligente Einsatzplanung** → Bedarfsgerechte Leerung
- ✓ **Automatisierte Prozesse** → Kosteneinsparung und Effizienz
- ✓ **Skalierbare Architektur** → Mehrtonnen-Netzwerke möglich

# Zielsetzung

„ Konzeption und Entwicklung einer intelligenten, **autonomen Mülltonne**, die durch moderne Sensorik und digitale Steuerungssysteme **selbstständig** ihren Füllstand erfasst, Hindernisse erkennt und durch Kommando eines Servers sich **eigenständig** entlang vorgegebener Wege bewegt. “

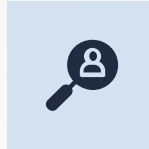
# Projektteam



## CHRISTOPH HARDES

**Schwerpunkt:**  
Konstruktion & Mechanik

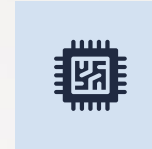
- 3D-Konstruktion (Inventor)
- CAD-Modellierung des Chassis
- Mechanischer Zusammenbau & Optimierung



## PHILIPP ZAJACZKOWSKI

**Schwerpunkt:** User-  
Interface Design & Elektronik

- Gestaltung des Feedbacks für den User
- Schaltungsdesign und Komponentenauswahl
- Verdrahtung und Kabelführung

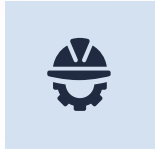


## DANIEL TÜBEN

**Schwerpunkt:** Programmierung  
& Software-Architektur

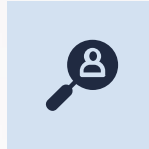
- MicroPython-Entwicklung (VS-Code)
- Zustandsmaschine-Logik (State Machine)
- Server-Client-Kommunikation
- Linienverfolgung (PD-Regler)

# Projektteam



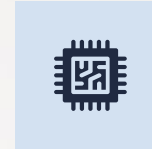
## CHRISTOPH HARDES

**Schwerpunkt:**  
Konstruktion & Mechanik



## PHILIPP ZAJACZKOWSKI

**Schwerpunkt:** User-  
Interface Design & Elektronik



## DANIEL TÜBEN

**Schwerpunkt:** Programmierung  
& Software-Architektur

## BETREUUNG:

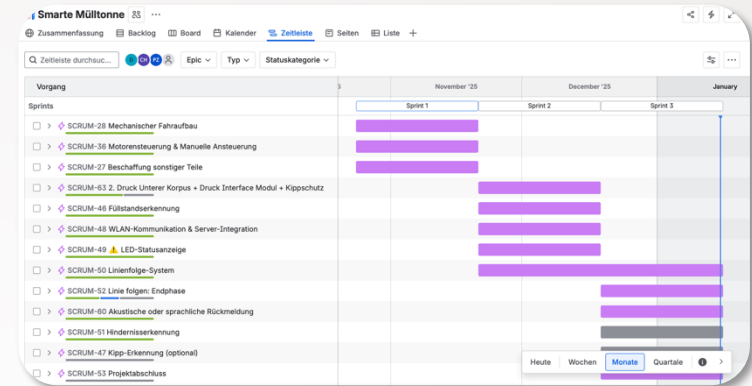
- Prof. Dr. Frank Hellweg (Software/Elektronik)
- Prof. Dr. Jens Bechthold (Mechanik/Hardware)
- Prof. Markus Strick (User Design/Usability)

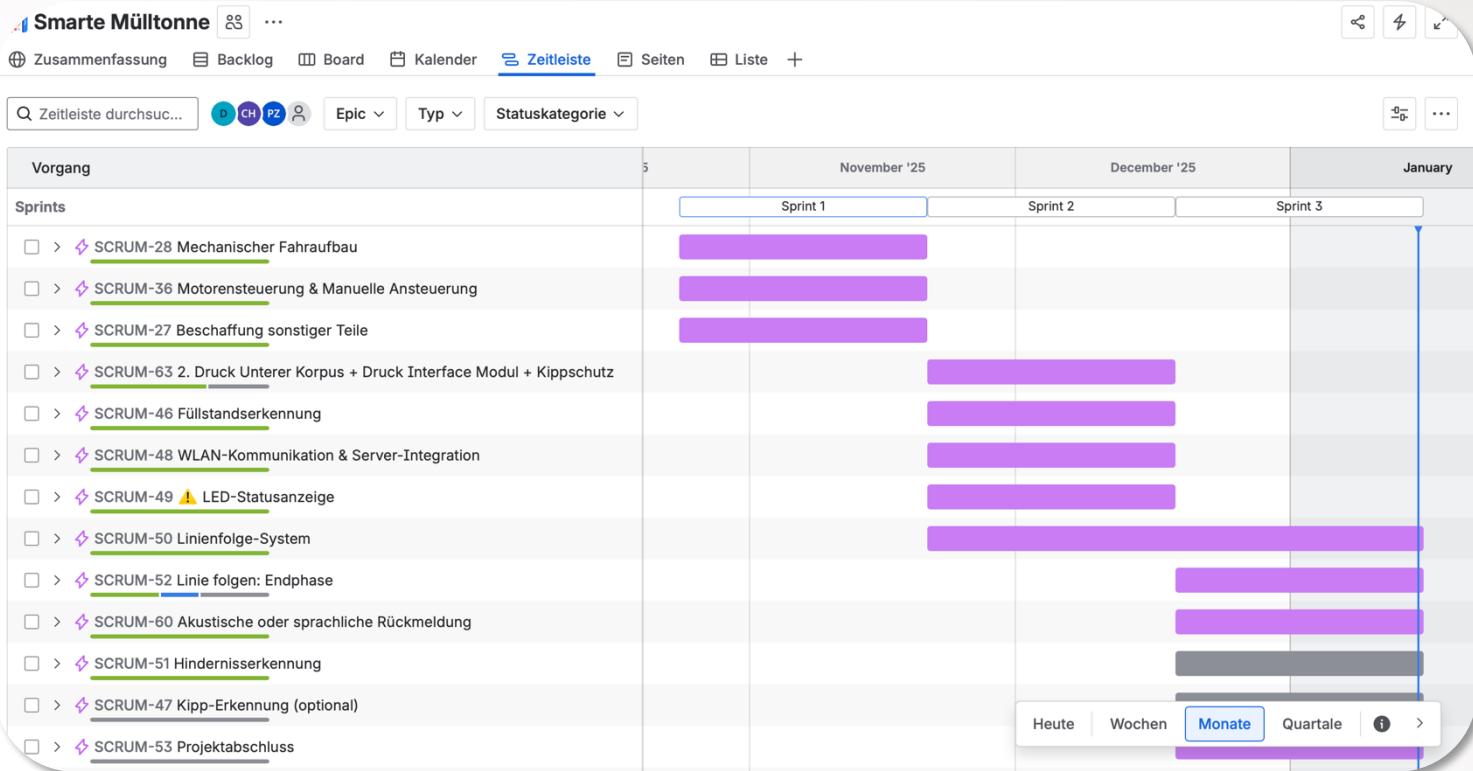
# PROJEKTORGANISATION: AGILE SCRUM-STRUKTUR

Das Projekt wurde nach **agilen Scrum-Prinzipien** organisiert mit:

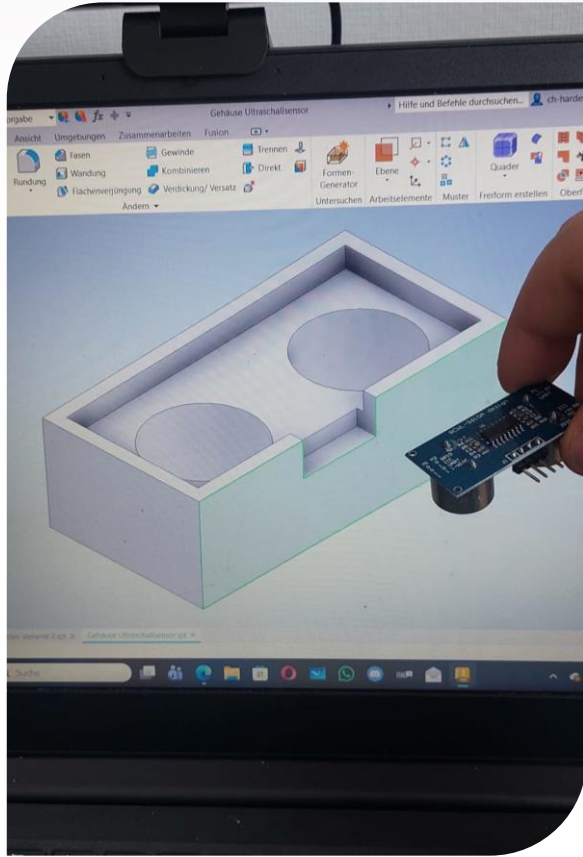
- 2-Wöchentliche Abstimmungsmeetings (freitags, 14:30 Uhr)
- Sprint-basierte Meilensteine (je 4 Wochen)
- Zentrales Issue-Tracking über Jira
- Dokumentation in Sciebo (geteilter Cloud-Speicher)
- Video-Meetings über Zoom

## DIE 3 SPRINTS: ZEITSTRUKTUR UND MEILENSTEINE:









1.

# KONSTRUKTION & MECHANIK

# GEHÄUSEDESIGN & 3D-KONSTRUKTION

## GEHÄUSEDESIGN

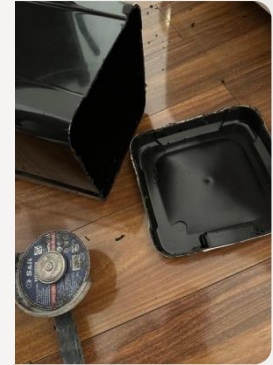
Basis: Handelsübliche 50-Liter-Mülltonne

### Anpassungen:

- Oberteil gekürzt (30 cm Tiefe nutzbares Volumen)
- Unterteil: 3D-gedruckter Montagerahmen für Fahrwerk
- CAD mit Inventor: Iterative Verbesserungen über 3 Sprints

### Material & Prozess:

- Kunststoff-Gehäuse (von Hand zugeschnitten mit Flex)
- 3D-Druck: Fahrwerk-Montagerahmen (PLA, ~4 Druckzyklen)
- Montage: Kleben + Verschraubung für maximale Stabilität



# INTERNE STRUKTUR

## BEDIENFELD (Oben)

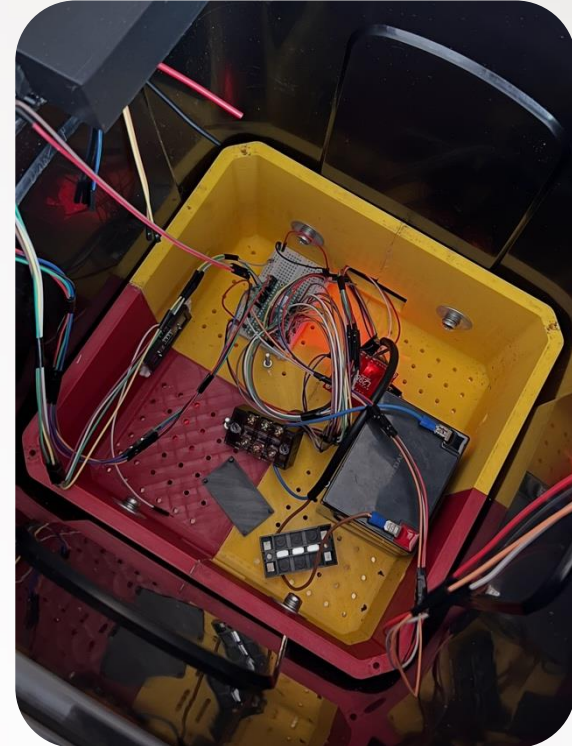
- LED-Status-Anzeige
- Taster (Grün/Rot)
- Kippschalter
- Ladeanschluss

## ELEKTRONIK-BEREICH

- Raspberry Pi Pico W
- L298N Motor-Treiber
- Blei-Akku (12V 5Ah)
- Spannungswandler (Laden)

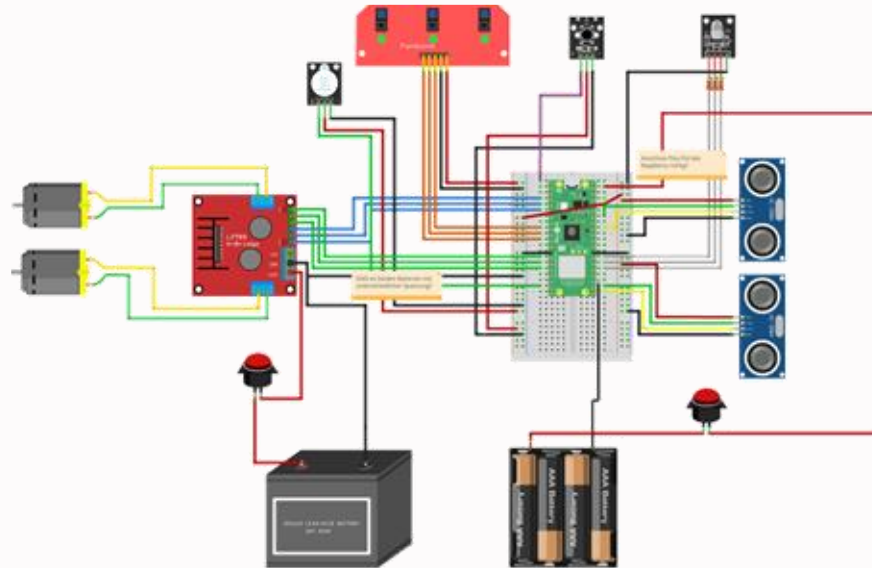
## SENSOR-MONTAGE

- Ultraschallsensor (oben und vorne)
- Liniensensoren (unten)



# INTERNE STRUKTUR

## ELEKTRISCHER SCHALTPLAN:



# ANTRIEB: RAUPENFAHRWERK

## WARUM RAUPENFAHRWERK UND NICHT RÄDER?

- Hohe Traktion auf allen Untergrund-Arten (Asphalt, Kopfstein)
- Große Auflagefläche → Geringer Bodendruck
- Stabiles Fahren auch bei Kurven und Unebenheiten
- Zuverlässige Hindernisüberwindung
- Gut für Gewichtslast-Verteilung
- **Nachteil:** Langsamer, höherer Stromverbrauch

## MOTOR-SPEZIFIKATIONEN

### Original-Motoren (Einsatzversion):

- Modell: 33GB-520 DC-Motoren
- Spannung: 6–12 V
- Leistung: ~0,5 W pro Motor (bei 12V)
- Drehzahl: ~100 U/min (theoretisch)

### Problem erkannt (Sprint 2):

- Bei 70% Motorlast fährt Tonne überhaupt nicht (zu schwer!)
- Gewicht + Reibung > Motorleistung

### Bessere Motoren (Sprint 3):

- 100 U/min, höheres Drehmoment
- Endgeschwindigkeit: 0,21 m/s (0,75 km/h)
- LANGSAMER aber SICHER

# ANTRIEB: RAUPENFAHRWERK

## MOTOR-SPEZIFIKATIONEN

### Original-Motoren (Einsatzversion):

- Modell: 33GB-520 DC-Motoren
- Spannung: 6–12 V
- Leistung: ~0,5 W pro Motor (bei 12V)
- Drehzahl: ~100 U/min (theoretisch)

### Problem erkannt (Sprint 2):

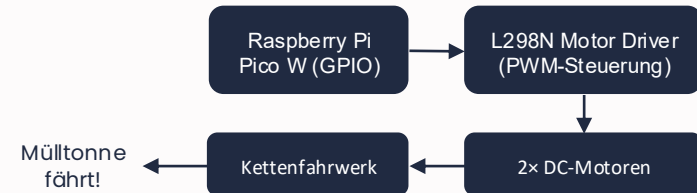
- Bei 70% Motorlast fährt Tonne überhaupt nicht (zu schwer!)
- Gewicht + Reibung > Motorleistung

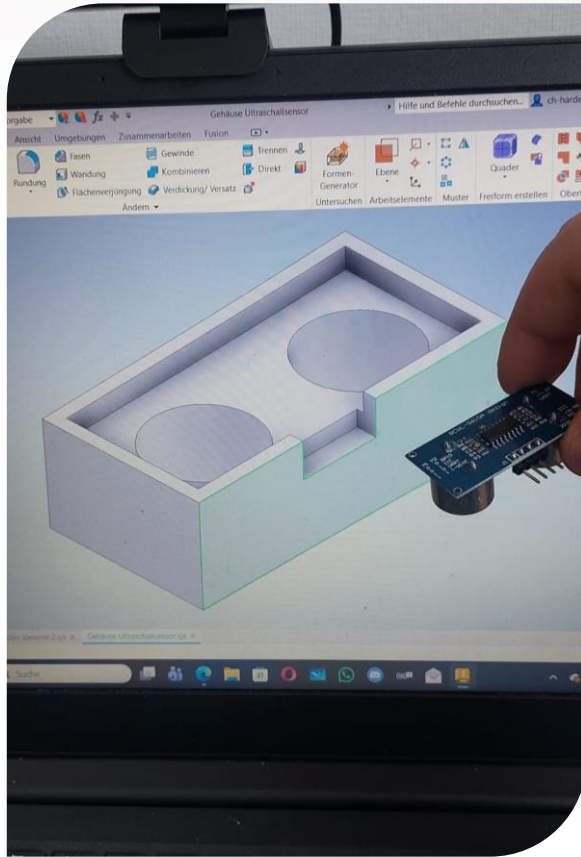
### Bessere Motoren (Sprint 3):

- 100 U/min, höheres Drehmoment
- Endgeschwindigkeit: 0,21 m/s (0,75 km/h)
- LANGSAMER aber SICHER



### MOTORSTEUERUNG (L298N Dual H-Bridge) Architektur:





1.

# KONSTRUKTION & MECHANIK



# 2.

## User-Interface & Sensorik

**BENUTZERSCHNITTSTELLE**

**STATUSANZEIGE**



# BEDIENFELD – BENUTZERSCHNITTSTELLE



Das System wird über drei intuitive Bedienelemente gesteuert:

- **Grüner Starttaster** – Startet die Fahrt zur Straße oder zurück zum Haus
- **Roter Stopptaster** – Sofort-Stopp und Rücksetzen des Systems
- **Hauptschalter** – Aktiviert und deaktiviert das gesamte System

# BEDIENFELD – STATUSANZEIGE

## LED-Statusanzeigen (RGB-LEDs)

Das System meldet seinen Status durch farbcodierte LED-Signale zurück:

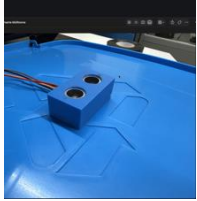
- **GRÜN:** Normalbetrieb, Standby-Modus, bereit für Befehle
- **GELB:** Mülltonne ist voll
- **BLAU:** Fahrt- oder Pausen-Modus aktiv (Button wurde gedrückt)
- **ORANGE:** Hindernis erkannt – Automatische Vermeidung läuft

## Zusätzlich Akustisches Feedback:

Buzzer signalisiert kritische Ereignisse (Fehler, Hindernisse, Systemwechsel)

| Zustands - LED                                                                                                                                                       | Server - LED                                                                                                                                                               |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p> <b>Grün (konstant)</b><br/>Bereit / Standby – Mülltonne wartet auf Einsatz</p>  | <p> <b>Grün (konstant)</b><br/>Verbunden – Server-Kommunikation aktiv</p>                 |
| <p> <b>Blau (pulsierend)</b><br/>In Bewegung – Mülltonne fährt oder dreht sich</p>  | <p> <b>Blau (schnell blinkend)</b><br/>Verbindet – stellt Verbindung her</p>              |
| <p> <b>Gelb (konstant)</b><br/>Voll – Mülltonne muss geleert werden</p>             | <p> <b>Lila (konstant)</b><br/>Offline-Modus – absichtlich getrennt während der Fahrt</p> |
| <p> <b>Gelb (pulsierend)</b><br/>Wartet auf Abholung – an der Straße angekommen</p> |                                                                                                                                                                            |
| <p> <b>Grün (pulsierend)</b><br/>Entleert – wurde erfolgreich geleert</p>           |                                                                                                                                                                            |
| <p> <b>Blau (konstant)</b><br/>Pausiert – Fahrt wurde unterbrochen</p>              |                                                                                                                                                                            |
| <p> <b>Orange (Doppelblitz)</b><br/>Hindernis erkannt – Fahrt gestoppt</p>          |                                                                                                                                                                            |
| <p> <b>Grün (blinkend)</b><br/>Leerfahrt geplant – wartet auf Serverbestätigung</p> |                                                                                                                                                                            |
| <p> <b>Gelb (blinkend)</b><br/>Vollfahrt geplant – wartet auf Serverbestätigung</p> |                                                                                                                                                                            |

# SENSOR-SUITE – FUNKTIONSBESCHREIBUNG



## FÜLLSTANDS- SENSOR

Typ: Ultraschallsensor für Abstandsmessung

Funktion: Kontinuierliche Messung des Müll-Volumens

Output: Prozentuale Füllstandsanzeige

Integration: Sendet Alarm an Server, wenn Tonne 90%+ voll ist



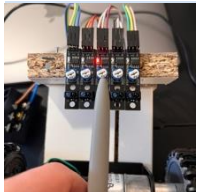
## HINDERNIS- ERKENNUNG

Typ: Ultraschallsensor für Abstandsmessung

Funktion: Erkennt Objekte vor der Mülltonne in Echtzeit

Reichweite: Bis zu 2-3 Meter

Verhalten: Bei Erkennung → Automatische Notbremse



## LINIEN- SENSOREN

Anzahl: 5 digitale Infrarot-Sensoren in linearer Anordnung

Funktion: Erkennung schwarzer Linien auf hellem Untergrund

Positionsberechnung: Gewichtetes 5-Punkt-System (-150 bis +150)

Präzision: Ermöglicht Navigation mit Millimeter-Genauigkeit



# 2.

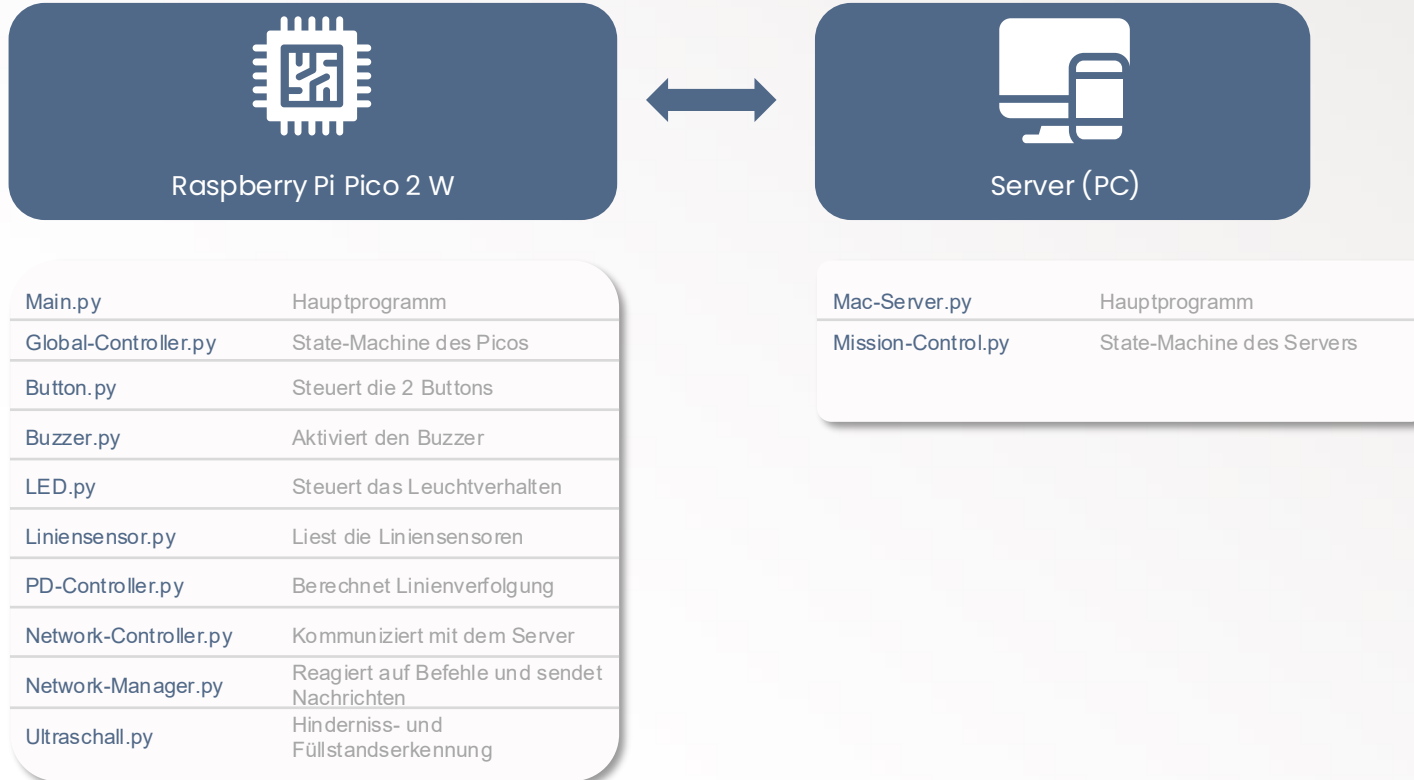
## User-Interface & Sensorik

# 3.

## Programmierung & Steuerungslogik

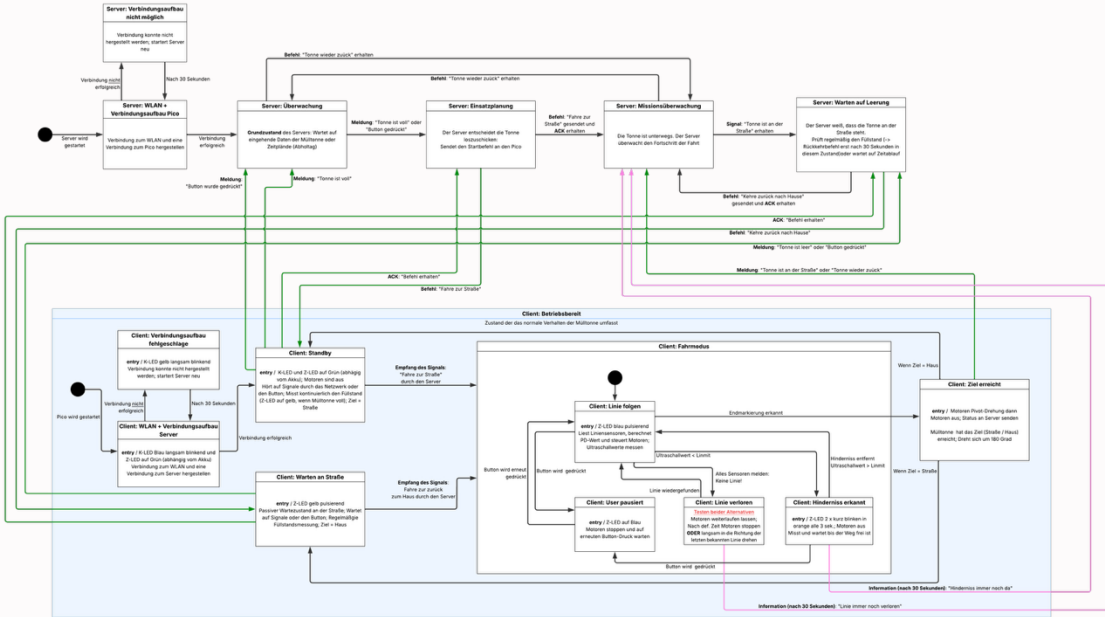
```
global_controller.py 6, M X main.py M liniensensor.py M
global_controller.py > GlobalController > __init__
6 class GlobalController:
801 def _handle_empty_reporting(self):
802
803     if self._empty_reported:
804         return
805
806     # Candidate starten
807     if not self._empty_candidate:
808         if is_empty:
809             self._empty_candidate = True
810             self._empty_first_ms = now
811             self._empty_debug_last_ms = 0
812             print(f"[DEBOUNCE][EMPTY] Start: {self.fuellstand_prozent}% (<= {self.EMPTY_CONFIRM_MS})")
813             return
814
815     # Candidate läuft: wenn Bedingung nicht mehr stimmt -> abbrechen (mit Debug)
816     if not is_empty:
817         print(f"[DEBOUNCE][EMPTY] Abbruch: {self.fuellstand_prozent}% (> {self.EMPTY_CONFIRM_MS})")
818         self._empty_candidate = False
819         return
820
821     # Candidate läuft: während Entprellzeit regelmäßig Debug ausgeben
822     elapsed = time.time_diff(now, self._empty_first_ms)
823     if self._empty_debug_last_ms == 0 or time.time_diff(now, self._empty_debug_last_ms) > self.EMPTY_DEBUG_INTERVAL:
824         remaining = self.EMPTY_CONFIRM_MS - elapsed
825         if remaining < 0:
826             remaining = 0
827         print(f"[DEBOUNCE][EMPTY] {self.fuellstand_prozent}% (<= {self.EMPTY_CONFIRM_MS}) {remaining}ms")
828         self._empty_debug_last_ms = now
829
830     # Bestätigung nach Entprellzeit
831     if elapsed >= self.EMPTY_CONFIRM_MS:
832         self._empty_reported = True
833         self._empty_candidate = False
834         print(f"[DEBOUNCE][EMPTY] Bestätigt: {self.fuellstand_prozent}% -> STATE_EMPTY")
835         self.set_pico_state(self.STATE_EMPTY)
836
837 def follow_line(self):
838     if not self.sensor_array or not self.pd_controller:
839         return False
840
841     position = self.sensor_array.get_position()
842
843     if position is None:
844         self.no_line_count += 1
845         if self.no_line_count > self.NO_LINE_NEEDED:
846             return False
```

# SOFTWARE-ARCHITEKTUR – MODULARE STRUKTUR



# ZUSTANDSMASCHINE - SERVER/CLIENT

State-Machine Server →



State-Machine Pico →

# ZUSTANDSMASCHINE – SERVER/CLIENT

## Server-Zustand

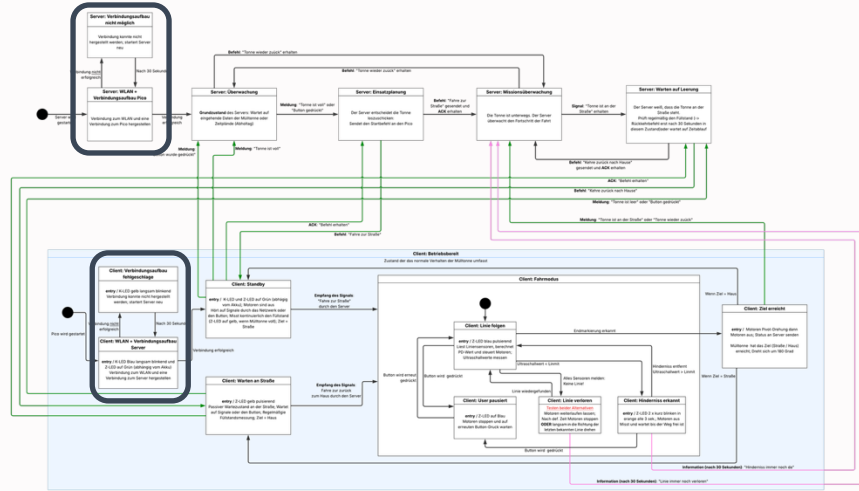
### Verbindungsaufbau zum Client:

- Wartet auf eingehende Verbindungen
- LED blinkt Blau (nach erfolgreicher Verbindung Grün)

## Pico-Zustand

### Verbindungsaufbau WLAN + Server:

- Verbindet sich mit WLAN
- Bei **erfolgreicher** WLAN-Verbindung: TCP-Verbindung zum Server
- Verbindungsaufbau **nicht möglich**: Pico wechselt in Offline-Modus (Client-LED → Lila)
  - Programm wird dann durch manuellen Buttondruck durchgeführt



# ZUSTANDSMASCHINE – SERVER/CLIENT

## Server-Zustand

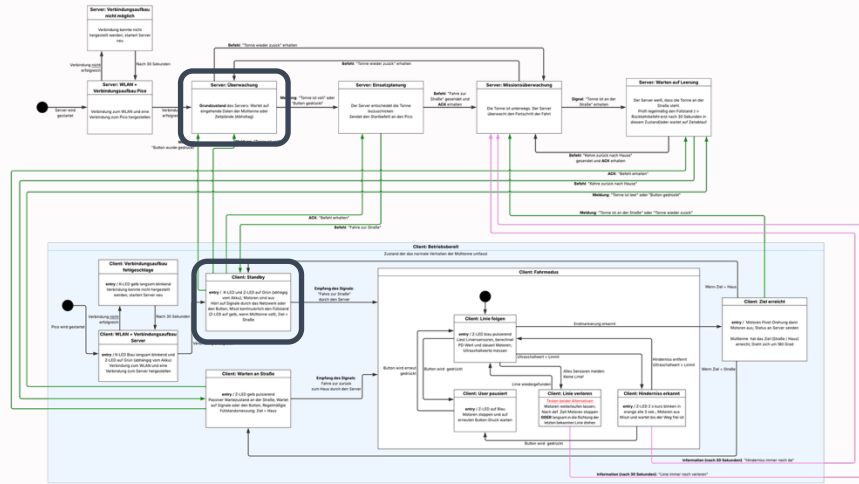
### Überwachung (Grundzustand):

- Wartet auf eingehende Daten des Clients (Voll / Button wurde gedrückt)

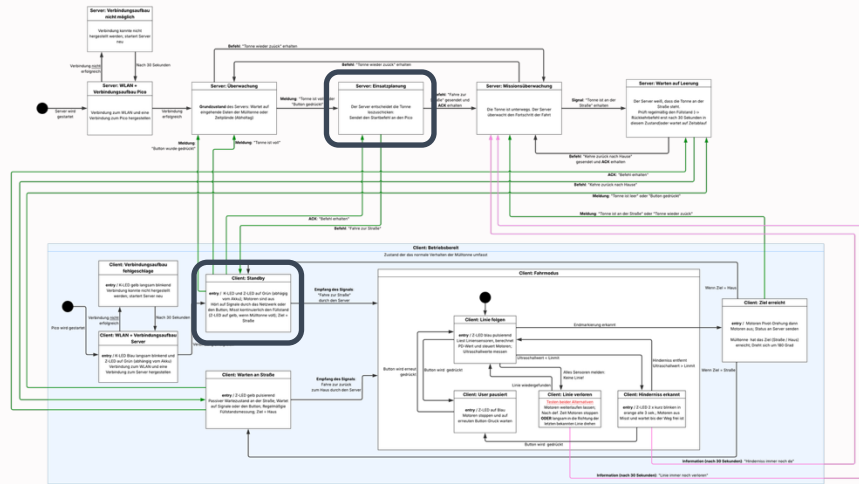
## Pico-Zustand

### Standby (Grundzustand):

- Misst kontinuierlich den Füllstand und prüft den grünen Button
- Bei Füllstand größer 80% oder kurzen Drücken des grünen Buttons, Signal an den Server
- Client-LED leuchtet Grün (Gelb, wenn Tonne voll; Grün/Gelb blinkend bei Buttondruck)



## ZUSTANDSMASCHINE - SERVER/CLIENT



## Server-Zustand

### Einsatzplanung:

- Ist die Tonne voll oder wurde der Button gedrückt, plant der Server die Mission
- Zukünftig: Abgleich mit Müllplan

## Pico-Zustand

### Standby (Grundzustand):

- Wartet auf Los-Kommando des Servers

# ZUSTANDSMASCHINE – SERVER/CLIENT

## Server-Zustand

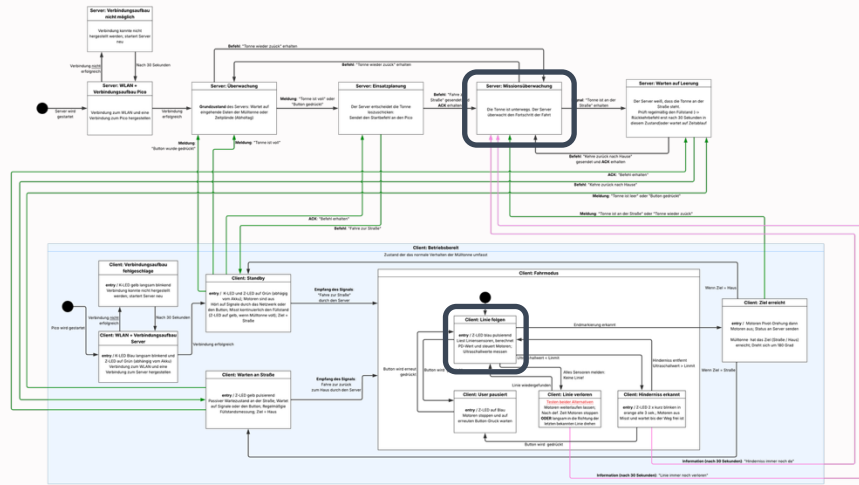
### Missionsüberwachung:

- Überwacht den Fortschritt der Fahrt

## Pico-Zustand

### Linie folgen:

- Trennt die Verbindung zum Server
- Ablauf der Fahrt:
  - Liest Werte der Liniensensoren aus
  - Berechnet durch PD-Regler die Korrektur
  - Gibt die Korrektur an die Motoren weiter
- Überwacht Hindernissensor und roten Button
- Client-LED pulsiert Blau



# ZUSTANDSMASCHINE – SERVER/CLIENT

## Server-Zustand

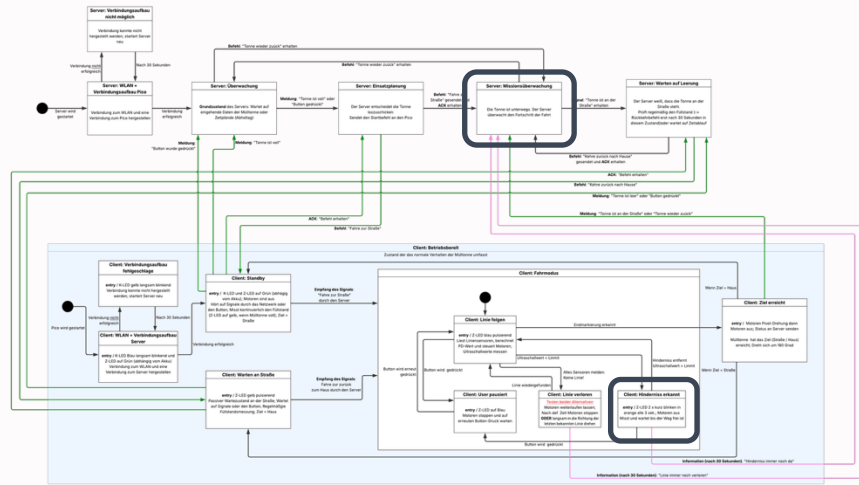
### Missionsüberwachung:

- Überwacht den Fortschritt der Fahrt
- Erhält die Meldung, dass ein Hindernis im Weg ist

## Pico-Zustand

### Hindernis erkannt:

- Stellt Verbindung zum Server her
- Misst kontinuierlich die Distanz zum Hindernis
- Client-LED blinkt Orange und Buzzer piept



# ZUSTANDSMASCHINE – SERVER/CLIENT

## Server-Zustand

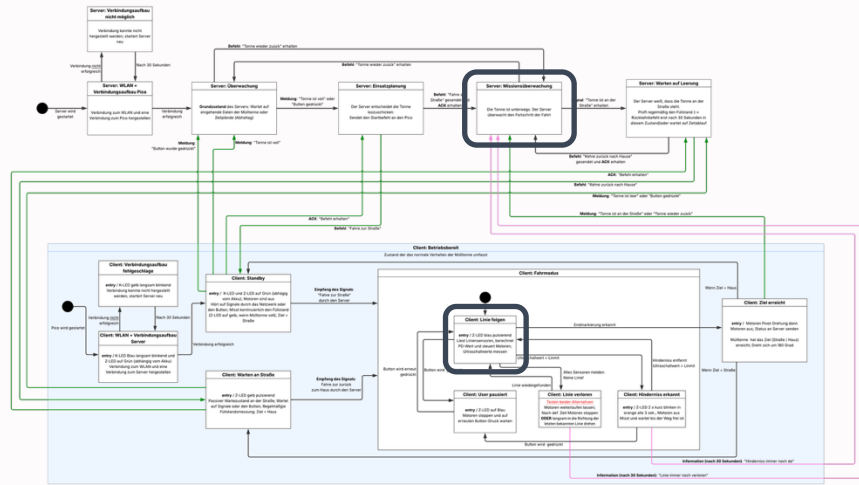
### Missionsüberwachung:

- Überwacht den Fortschritt der Fahrt

## Pico-Zustand

### Linie folgen:

- Trennt die Verbindung zum Server
- Ablauf der Fahrt:
  - Liest Werte der Liniensensoren aus
  - Berechnet durch PD-Regler die Korrektur
  - Gibt die Korrektur an die Motoren weiter
- Überwacht Hindernissensor und roten Button
- Client-LED pulsiert Blau



# ZUSTANDSMASCHINE – SERVER/CLIENT

## Server-Zustand

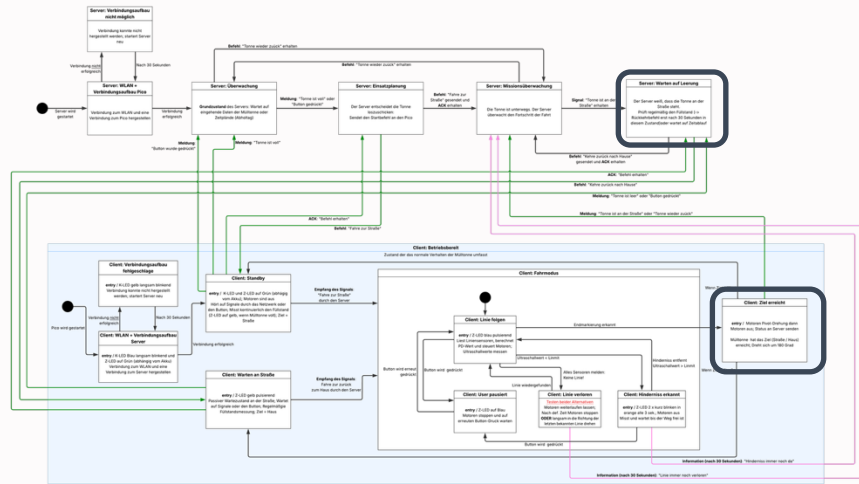
### Warten auf Leerung:

- Wartet auf eingehende Signale des Clients (Voll / Button wurde gedrückt)

## Pico-Zustand

### Ziel erreicht:

- Stellt Verbindung zum Server her
- Stoppt die Motoren und führt anschließend eine 180-Grad Drehung durch
- Informiert den Server über die Ankunft
- Client-LED pulsiert Blau



# ZUSTANDSMASCHINE – SERVER/CLIENT

## Server-Zustand

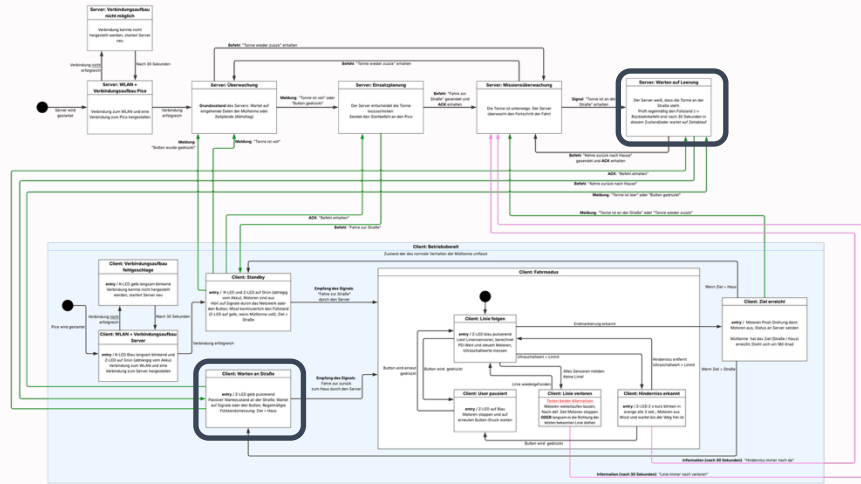
### Warten auf Leerung:

- Wartet auf eingehende Signale des Clients (Voll / Button wurde gedrückt)

## Pico-Zustand

### Wartet an der Straße:

- Misst kontinuierlich den Füllstand und prüft den grünen Button
- Bei Füllstand weniger 20% oder kurzen Drücken des grünen Buttons, Signal an den Server
- Client-LED pulsiert Gelb (Grün, wenn Tonne leer; Grün/Gelb blinkend bei Buttondruck)



# ZUSTANDSMASCHINE – SERVER/CLIENT

## Server-Zustand

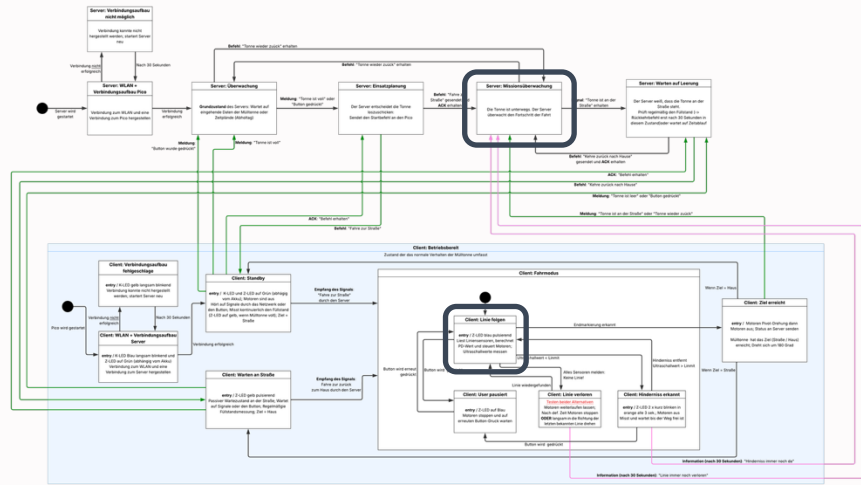
### Missionsüberwachung:

- Überwacht den Fortschritt der Fahrt

## Pico-Zustand

### Linie folgen:

- Trennt die Verbindung zum Server
- Überwacht Hindernissensor und roten Button
- Client-LED pulsiert Blau



# ZUSTANDSMASCHINE – SERVER/CLIENT

## Server-Zustand

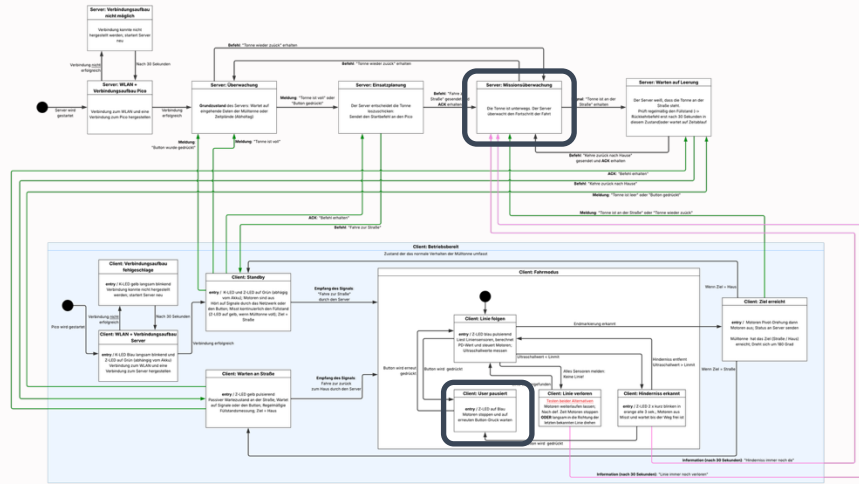
### Missionsüberwachung:

- Überwacht den Fortschritt der Fahrt
- Erhält die Meldung, dass der User pausiert hat

## Pico-Zustand

### User pausiert:

- Stellt Verbindung zum Server her
- Wartet bis der grüne Button gedrückt wird
- Client-LED leuchtet Blau



# ZUSTANDSMASCHINE – SERVER/CLIENT

## Server-Zustand

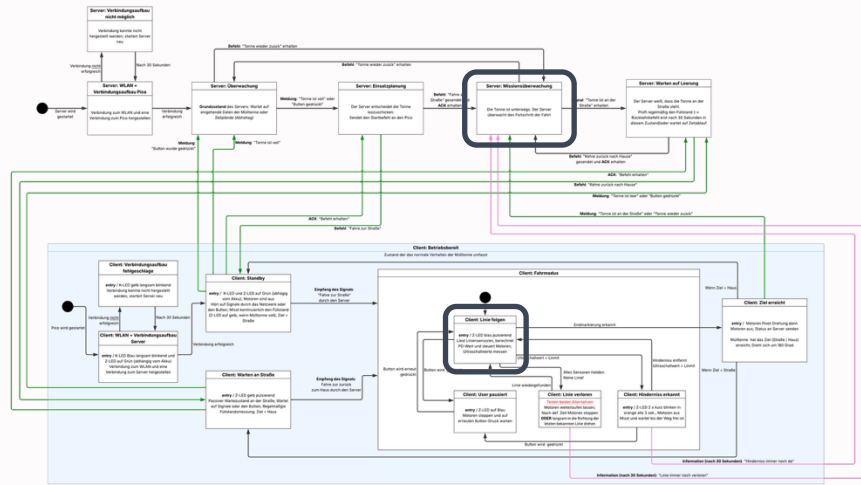
### Missionsüberwachung:

- Überwacht den Fortschritt der Fahrt

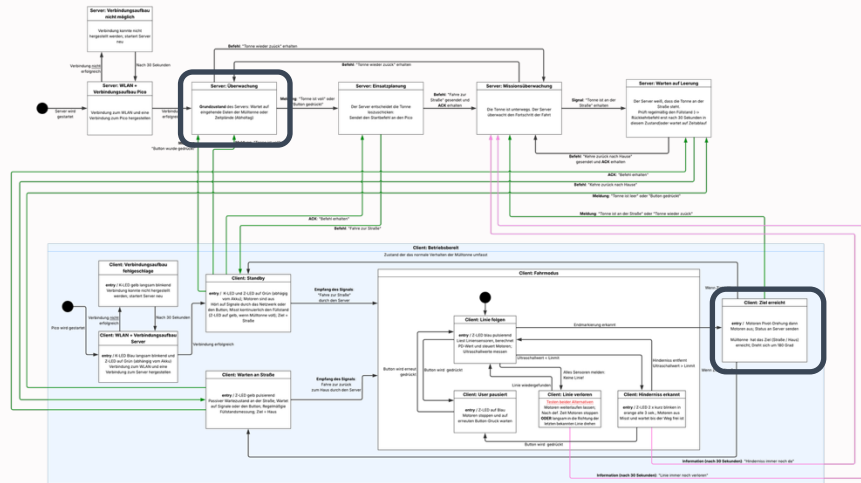
## Pico-Zustand

### Linie folgen:

- Trennt die Verbindung zum Server
- Überwacht Hindernissensor und roten Button
- Client-LED pulsiert Blau



## ZUSTANDSMASCHINE - SERVER/CLIENT



## Server-Zustand

### Überwachung (Grundzustand):

- Wartet auf eingehende Daten des Clients (Voll / Button wurde gedrückt)

## Pico-Zustand

**Ziel erreicht:**

- Stellt Verbindung zum Server her
- Stoppt die Motoren und führt anschließend eine 180-Grad Drehung durch
- Informiert den Server über die Ankunft
- Client-LED pulsiert Blau



# AUSBLICK – ZUKÜNFTIGE AUSBAUSTUFEN

## Multi-Tonnen-Netzwerk

Mehrere autonome Mülltonnen mit  
Routenplanung

→ Zentrale Verwaltung und Optimierung der  
Abfallwirtschaft

## Machine Learning

Lernfähige Fahrmuster und Hinderniserkennung

→ Selbstoptimierung durch gesammelte  
Einsatzdaten

## Upgrade Sensoren

Austausch der Liniensensoren durch Sensoren, die  
kontinuierliche Werte ausgeben

→ Verbessert das Linienverhalten und macht das  
Fahren flüssiger

## Mobile App

Echtzeit-Benachrichtigungen und Remote-  
Steuerung

→ Nutzer erhält Statusmeldungen auf dem  
Smartphone

## Müllkalender

Server erhält einen Kalender der Müllabholung  
und Mülltonne wird erst am Abholtag zur Straße  
geschickt

→ Effiziente Fahrten

# Vielen Dank!

Für Ihre Zeit und Aufmerksamkeit.